

Data Engineer Program Edición 33

5. Procesamiento en Tiempo Real con Apache Kafka y Spark
- Procesamiento en Tiempo Real con Spark y Kafka

Docker, Kafka y Spark: aprendizajes sobre el procesamiento de datos en tiempo real

Introducción

Los ejercicios de frases aleatorias, posición satelital y sensores IoT permitieron comprender cómo se integran distintas tecnologías en una solución de procesamiento de datos en tiempo real. Docker proporcionó un entorno controlado para ejecutar los servicios; Kafka transportó los eventos de forma continua entre productores y consumidores; y Spark procesó, transformó y analizó la información recibida.

La práctica avanzó de manera progresiva: comenzó con la transmisión básica de mensajes, continuó con datos estructurados y culminó en un escenario cercano a una aplicación real de Internet de las Cosas (IoT), en el que las mediciones de sensores se analizan para generar alertas oportunas.

1. Docker: entorno reproducible y aislado

El principal aprendizaje fue comprender el concepto de **contenedor** y su utilidad para crear entornos de desarrollo reproducibles. Docker permite ejecutar de forma aislada los componentes de una solución y evita instalar cada dependencia directamente en el sistema operativo.

En los ejercicios, la arquitectura se desplegó mediante contenedores para Kafka, Zookeeper, Spark Master, Spark Worker y Jupyter.

El archivo **docker-compose.yml** centralizó la definición de servicios, redes, puertos, dependencias y volúmenes. Así, todo el entorno pudo iniciarse de manera consistente con una sola configuración.

También se comprendió la importancia de los **volúmenes**, ya que permiten compartir archivos entre el computador y los contenedores y conservar información fuera del ciclo de vida de un contenedor. Esto fue particularmente importante para trabajar con los notebooks y los resultados generados durante el procesamiento.

Otro aprendizaje importante fue entender que los contenedores no son máquinas virtuales. Un contenedor es más ligero porque comparte el kernel del sistema operativo anfitrión, mientras que una máquina virtual requiere un sistema operativo completo.

2. Kafka: transmisión de eventos

Kafka permitió comprender el **procesamiento basado en eventos**. En lugar de depender únicamente de archivos estáticos, un productor genera información de forma continua y la publica en un tema (*topic*) para que otros componentes la consuman.

La arquitectura utilizada siguió este flujo:

Producer → Kafka → Consumer

El productor genera y publica mensajes en un *topic*; el consumidor se suscribe a ese canal y recibe los eventos para su procesamiento.

Progresión de los ejercicios. El caso de frases aleatorias introdujo el mecanismo básico: el productor enviaba mensajes —por ejemplo, «procesamiento en tiempo real»— a Kafka y Spark los consumía.

El ejercicio de posición satelital incorporó datos estructurados que Kafka transportaba y Spark interpretaba. Después, el caso de sensores llevó la arquitectura a un escenario IoT con mediciones de temperatura, humedad y presión comparadas con valores de referencia.

Conceptos esenciales. Los *topics* organizan los flujos de eventos; las particiones distribuyen los mensajes y habilitan el procesamiento paralelo; y el factor de replicación mantiene copias de las particiones para mejorar la tolerancia a fallos.

3. Spark: procesamiento y análisis distribuido

Apache Spark permitió comprender el papel de un motor de procesamiento distribuido dentro de una arquitectura de datos. La práctica se concentró en dos componentes: **DataFrames** y **Spark Structured Streaming**.

DataFrames. Organizan la información en filas y columnas y facilitan operaciones de selección, transformación, filtrado, unión y agregación.

Structured Streaming. Procesa los mensajes a medida que llegan desde Kafka, sin esperar a que exista un archivo completo para iniciar el análisis.

Aplicación con sensores. Los mensajes JSON se convirtieron en estructuras de Spark y se compararon con los umbrales definidos para cada máquina. El resultado clasificó cada medición como **NORMAL**, **ALERTA** o **CRÍTICO**.

Los resultados se mostraron en consola y se almacenaron en formato **Parquet** para facilitar consultas posteriores.

4. Desafíos y soluciones

La práctica mostró que la integración de tecnologías también genera dificultades de configuración, formato y coordinación, no solo problemas relacionados con la lógica del programa.

4.1. Caracteres no imprimibles

Uno de los primeros problemas apareció al copiar código Python al notebook. Algunos espacios no eran espacios normales, sino caracteres Unicode U+00A0.

Esto generaba errores de sintaxis como: `invalid non-printable character U+00A0`

La solución consistió en reemplazar los espacios especiales por espacios normales.

Este problema permitió comprender que el código aparentemente idéntico puede contener caracteres invisibles que afectan su ejecución.

4.2. Lectura y escritura en Parquet

Otro desafío surgió al intentar abrir directamente un archivo **.parquet** desde el explorador de Jupyter.

El mensaje «no está codificado en UTF-8» no indicaba que el archivo estuviera dañado. Parquet es un formato binario y, por tanto, no debe abrirse como texto.

La solución puede ser cargarlo con Spark mediante **`spark.read.parquet(...)`** y visualizar los datos con **`df_parquet.show()`**.

Esto permitió comprender mejor la diferencia entre un archivo de texto y un formato columnar binario utilizado para almacenamiento y análisis de datos.

4.3. Coordinación entre productor y consumidor

En los ejercicios también fue necesario coordinar correctamente el momento en que se inicia el Producer y el Consumer.

Cuando se utiliza: `startingOffsets = "latest"` el consumidor comienza a procesar los nuevos eventos que llegan después de iniciar el streaming.

Por esta razón, en el ejercicio de sensores se estableció explícitamente el flujo:

1. Iniciar el Consumer.
2. Preparar el streaming.
3. Iniciar el Producer.
4. Generar los eventos.
5. Observar las notificaciones.
6. Detener el streaming.
7. Leer los resultados almacenados en Parquet.

Esta secuencia permitió comprender que en arquitecturas de streaming el orden y el estado de los componentes pueden ser tan importantes como el código.

5. Reflexiones sobre el procesamiento en tiempo real

La principal diferencia entre el procesamiento por lotes y el procesamiento en tiempo real radica en el momento del análisis. En un flujo por lotes, los datos se procesan después de reunir un conjunto completo; en tiempo real, los eventos se analizan mientras se generan, lo que permite reaccionar casi de inmediato.

En el ejercicio de sensores, por ejemplo, esperar hasta el final del día para revisar la temperatura de una máquina podría impedir una respuesta oportuna. Detectar de inmediato una medición anormal permite emitir una alerta y actuar antes de que el problema se agrave.

Por ello, el procesamiento en tiempo real no consiste solo en mover datos con rapidez, sino en **convertir eventos en información útil mientras ocurren**.

6. Aplicaciones profesionales

La combinación de Docker, Kafka y Spark puede aplicarse en distintos sectores:

6.1. Sector financiero

En una entidad financiera, Kafka puede utilizarse para transportar eventos relacionados con transacciones, movimientos de cuentas o interacciones de clientes.

Spark puede analizar esos eventos para identificar comportamientos anormales, generar indicadores y apoyar sistemas de detección de fraude.

6.2. Industria y manufactura

El ejercicio de sensores representa directamente un escenario industrial.

Sensores instalados en máquinas pueden enviar continuamente:

- Temperatura.
- Presión.
- Humedad.
- Vibración.
- Consumo energético.

Kafka puede transportar esos eventos y Spark puede analizarlos para detectar condiciones anormales y anticipar posibles fallas.

6.3. Telecomunicaciones

Las empresas de telecomunicaciones generan grandes cantidades de eventos relacionados con llamadas, conexiones, tráfico de red y comportamiento de los usuarios.

El procesamiento en tiempo real permite detectar problemas de infraestructura, congestión y comportamientos anómalos.

6.4. Comercio electrónico

En una plataforma de comercio electrónico, los eventos generados por los usuarios pueden utilizarse para analizar inmediatamente:

- Visitas.
- Búsquedas.
- Carritos abandonados.
- Compras.
- Cambios en los productos.

Estos datos pueden alimentar sistemas de recomendación, análisis de comportamiento y monitoreo del negocio.

Parte 2: Actividad práctica con Docker + Kafka + Spark

A. Se levantan los servicios y se crea topic con las siguientes instrucciones:

- ✓ `docker-compose down -v`
- ✓ `docker-compose pull`
- ✓ `docker-compose up -d`
- ✓ `docker exec -it kafka bash`
- ✓ `kafka-topics --bootstrap-server kafka:9092 --list`
- ✓ `kafka-topics \--create \--topic actividad-topic \--bootstrap-server kafka:9092 \--partitions 1 \--replication-factor 1`
- ✓ Y desde la terminal en Jupiter se ejecuta `pip install kafka-python`

```
(base) jovyan@67cafe7cf5bb:~$ pip install kafka-python
Collecting kafka-python
  Downloading kafka_python-3.0.11-py3-none-any.whl (614 kB)
    614.2/614.2 kB 1.6 MB/s eta 0:00:00
Installing collected packages: kafka-python
Successfully installed kafka-python-3.0.11
✓ (base) jovyan@67cafe7cf5bb:~$
```

B. Descripción de la solución construida con procesamiento en tiempo real

Objetivo: implementar un sistema de procesamiento de datos IoT en tiempo real utilizando Kafka y Spark Structured Streaming. Un Producer generará continuamente mediciones de temperatura, humedad y presión de diferentes sensores y las publicará en un topic de Kafka. Spark consumirá los eventos, interpretará los mensajes JSON, transformará los datos y calculará indicadores agregados por sensor. Además, identificará mediciones que superen determinados umbrales y almacenará los resultados procesados en formato Parquet.

Consideraciones

- ✓ El producer genera continuamente datos simulados entre 1 a tres segundos de temperatura, humedad y presión para diferentes sensores.
- ✓ El consumer lee los eventos, convierte los datos, detecta valores fuera de un rango establecido. La decisión de clasificar las alertas en NORMAL/ALERTA/CRÍTICO la hará Spark en el Consumer
- ✓ Para el ejercicio se tendrán **5 máquinas**
 - M001 → Sensor S001
 - M002 → Sensor S002
 - M003 → Sensor S003
 - M004 → Sensor S004
 - M005 → Sensor S005
- ✓ Cada sensor tendrá un umbral

Máquina	Sensor	Temp. máxima	Humedad máxima	Presión mínima
M001	S001	30 °C	70 %	1000 hPa
M002	S002	35 °C	75 %	1000 hPa
M003	S003	28 °C	65 %	1005 hPa
M004	S004	32 °C	80 %	995 hPa

M005	S005	30 °C	70 %	1005 hPa
------	------	-------	------	----------

- ✓ La ejecución del producer3 en la terminal es de la siguiente forma:

```
(base) jovyan@1a63e8e46bea:~$ python producer3.py
Conexión con Kafka exitosa
Iniciando productor de sensores...
Topic: actividad-topic
Enviado → M003 | T=21.21°C | H=74.87% | P=1012.67 hPa
Enviado → M005 | T=36.89°C | H=77.04% | P=1013.74 hPa
Enviado → M004 | T=33.48°C | H=84.43% | P=980.98 hPa
Enviado → M003 | T=30.0°C | H=66.3% | P=1010.67 hPa
Enviado → M005 | T=33.59°C | H=55.77% | P=992.91 hPa
Enviado → M005 | T=34.34°C | H=81.57% | P=1002.89 hPa
Enviado → M003 | T=29.29°C | H=73.71% | P=991.44 hPa
Enviado → M005 | T=35.69°C | H=74.37% | P=1013.74 hPa
Enviado → M003 | T=33.57°C | H=58.44% | P=1009.57 hPa
```

- ✓ En el consumer se ejecutan en un notebook los siguientes pasos:

- Importar librerías
- Crear SparkSession
- Definir el esquema de los sensores
- Conectar con Kafka
- Convertir el JSON recibido
- Definir los umbrales. La salida es la siguiente:

```
Umbrales definidos:
+-----+-----+-----+-----+
|maquina_id|temp_max|humedad_max|presion_min|
+-----+-----+-----+-----+
|      M001|    30.0|      70.0|    1000.0|
|      M002|    35.0|      75.0|    1000.0|
|      M003|    28.0|      65.0|    1005.0|
|      M004|    32.0|      80.0|     995.0|
|      M005|    30.0|      70.0|    1005.0|
+-----+-----+-----+-----+
```

- Cruzar sensores con sus umbrales
- Calcular las alertas
- Determinar el estado de las alertas: NORMAL / ALERTA / CRITICO
- Seleccionar el resultado final
- Preparar las rutas de salida
- iniciar el streaming. La salida es la siguiente:

```

=====
CONSUMER DE SENSORES INICIADO
=====

Kafka está esperando eventos...

>>> AHORA INICIAR EL PRODUCER <<<

Ejecutar el Producer de sensores en la terminal.
Déjar generar eventos durante unos segundos.

Los eventos aparecerán por pantalla automáticamente.
También serán almacenados en formato Parquet.

=====

```

- Mostrar las notificaciones por pantalla
- Guardar los resultados en Parquet
- verificar que los streams están activos
- Detener el consumer. La salida es la siguiente:

```

=====
STREAMING DETENIDO
=====

Los eventos recibidos han quedado almacenados en Parquet.

Ruta:
work/jupyter_storage/resultado_sensores

```

- Leer el Parquet. La salida es similar a la siguiente:

Parquet leído correctamente.

Cantidad total de registros:

22

- Mostrar las últimas 10 ejecuciones. La salida es similar a la siguiente:

```

=====
RESULTADO FINAL - ÚLTIMAS 10 EJECUCIONES
=====
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|sensor_id|maquina_id|timestamp |temperatura|humedad|presion|temp_max|humedad_max|presion_min|alerta_temperatura|alerta_humedad|alerta_presion|total_a|
|lertas|estado |tipo_alerta|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|S004 |M004 |1791072333|25.19 |74.37 |994.36 |32.0 |80.0 |995.0 |0 |0 |1 |1 |
|ALERTA |PRESION |
|S002 |M002 |1791072330|38.38 |85.77 |995.87 |35.0 |75.0 |1000.0 |1 |1 |1 |3 |
|CRITICO|MULTIPLE |
|S005 |M005 |1791072327|31.18 |84.13 |1002.68|30.0 |70.0 |1005.0 |1 |1 |1 |3 |
|CRITICO|MULTIPLE |
|S005 |M005 |1791072325|31.81 |72.38 |991.42 |30.0 |70.0 |1005.0 |1 |1 |1 |3 |
|CRITICO|MULTIPLE |
|S004 |M004 |1791072323|37.64 |76.31 |987.07 |32.0 |80.0 |995.0 |1 |0 |1 |2 |
|CRITICO|MULTIPLE |
|S003 |M003 |1791072320|21.71 |76.75 |1002.71|28.0 |65.0 |1005.0 |0 |1 |1 |2 |
|CRITICO|MULTIPLE |
|S002 |M002 |1791072318|29.15 |89.11 |992.63 |35.0 |75.0 |1000.0 |0 |1 |1 |2 |
|CRITICO|MULTIPLE |
|S002 |M002 |1791072315|42.06 |74.46 |1006.96|35.0 |75.0 |1000.0 |1 |0 |0 |1 |
|ALERTA |TEMPERATURA|
|S003 |M003 |1791072313|25.32 |71.59 |1012.57|28.0 |65.0 |1005.0 |0 |1 |0 |1 |
|ALERTA |HUMEDAD |
|S005 |M005 |1791072310|26.01 |84.48 |995.95 |30.0 |70.0 |1005.0 |0 |1 |1 |2 |
|CRITICO|MULTIPLE |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

- Resumen final. La salida es similar a la siguiente:

```
=====
                        RESULTADO FINAL DEL EJERCICIO
=====

Total de eventos procesados: 22

Eventos por estado:
+-----+-----+
| estado|count|
+-----+-----+
| ALERTA|    8|
| CRITICO|  10|
|  NORMAL|    4|
+-----+-----+

Ubicación de los resultados Parquet:
work/jupyter_storage/resultado_sensores
```